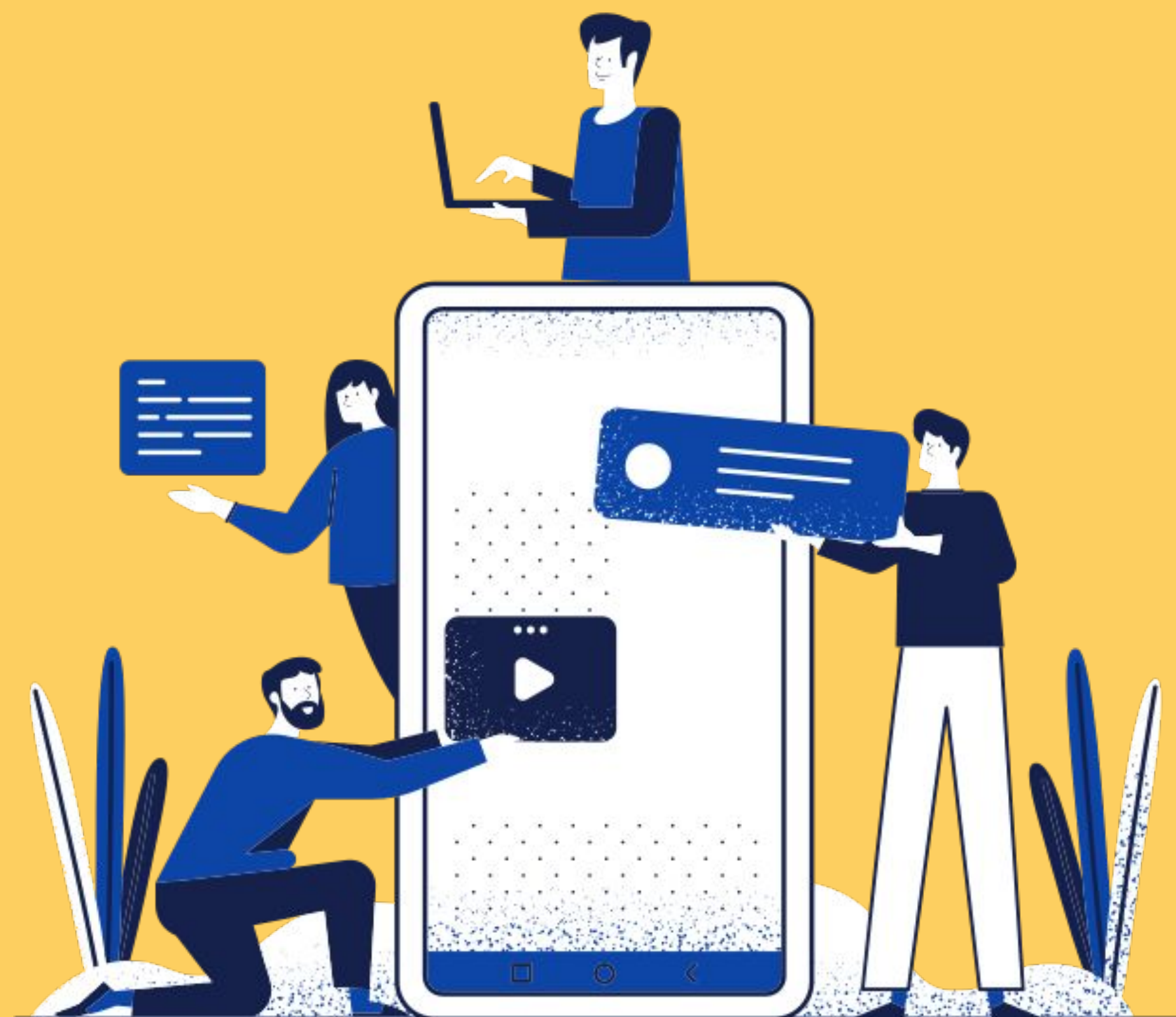
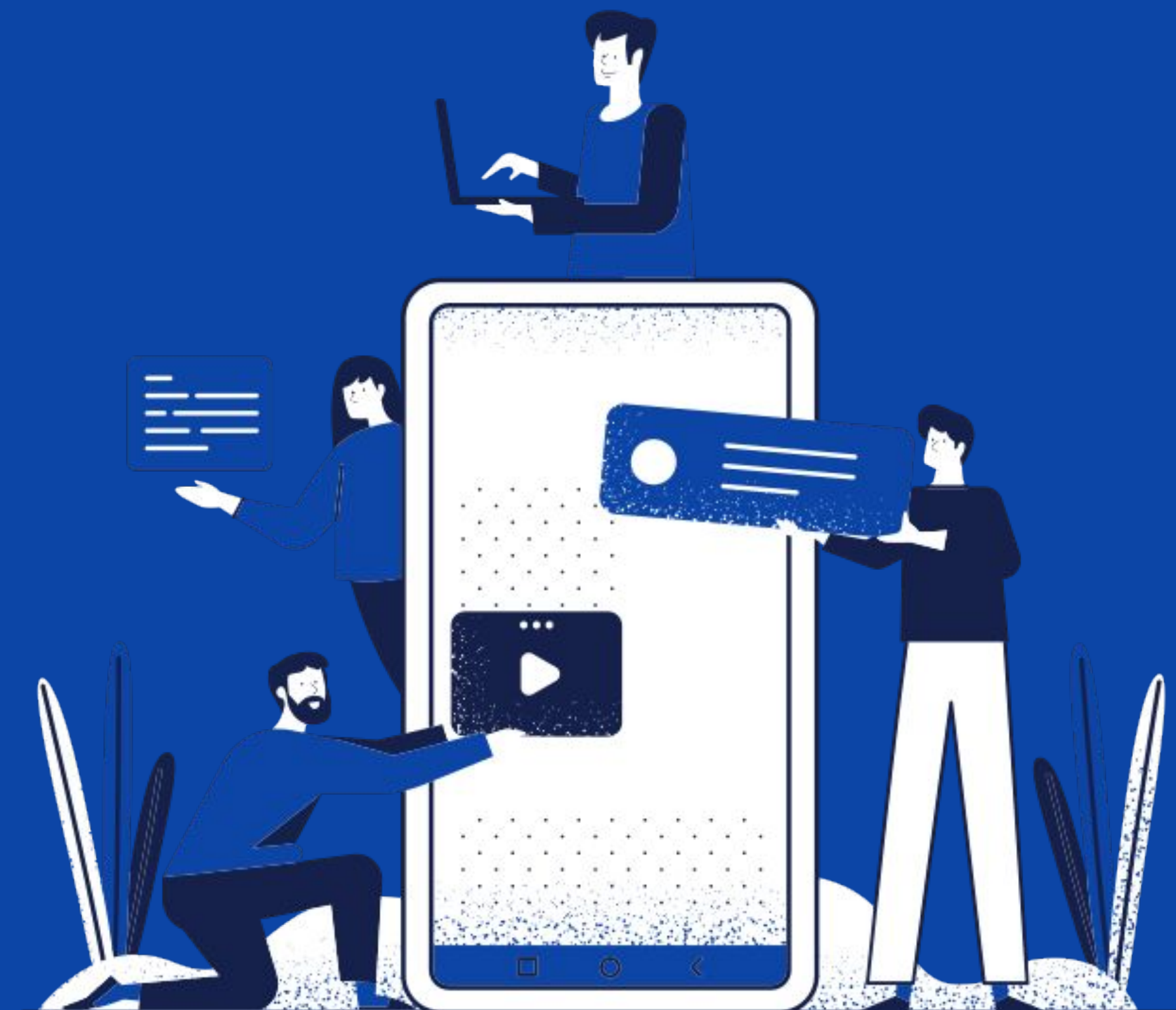






Fundamentos de Programação e Introdução ao Debugging com Node.js

**E voltamos a JavaScript e
Node.js...**

The screenshot shows a web browser window with a single tab titled 'Formulário elaborado'. The address bar shows a local file path: 'file:///D:/Projetos%20WEB/Projeto%20Curso'. The page title is 'Formulário'. The form is divided into two main sections: 'Dados pessoais' and 'Dados profissionais'. The 'Dados pessoais' section contains fields for 'Nome:', 'Endereço:', 'Cidade:', and 'Estado:' (with a dropdown menu showing 'São Paulo'). The 'Dados profissionais' section contains a 'Natureza do Cargo' section with radio buttons for 'Gerência', 'Financeiro', 'Recepção', 'Administrativo', and 'Jurídico'. Below this is an 'Área de interesse' section with checkboxes for 'Computação', 'Biologia', 'Meio Ambiente', 'Engenharia', and 'História'. At the bottom of the 'Dados profissionais' section is a 'Mini-curriculo:' label next to a large text area. At the very bottom of the form are two buttons: 'Enviar' and 'Limpar'.

- Surgiu em 1995.
- Criada por Brendan Eich em uma semana!
- Objetivo de ser uma linguagem fácil de usar para validar formulários HTML.
- Foi revolucionária na época.



**Java era uma linguagem
muito popular na época, então...**





JavaScript

- **JavaScript é uma linguagem de programação originalmente usada para o desenvolvimento de sites.**
- **As únicas linguagens que podem ser executadas no navegador são JavaScript e Web Assembly**

```

0000000000001169 <main>:
1169:    f3 0f 1e fa    endbr64
116d:    55             push    %rbp
116e:    48 89 e5       mov     %rsp,%rbp
1171:    ba 0c 00 00 00 mov     $0xc,%edx
1176:    48 8d 35 87 0e 00 00 lea     0xe87(%rip),%rsi    # 2004 <_IO_stdin_used+0x4>
117d:    bf 01 00 00 00 mov     $0x1,%edi
1182:    e8 d9 fe ff ff callq   1060 <write@plt>
1187:    bf 00 00 00 00 mov     $0x0,%edi
118c:    e8 df fe ff ff callq   1070 <exit@plt>
1191:    66 2e 0f 1f 84 00 00 nopw    %cs:0x0(%rax,%rax,1)
1198:    00 00 00
119b:    0f 1f 44 00 00 nopl    0x0(%rax,%rax,1)

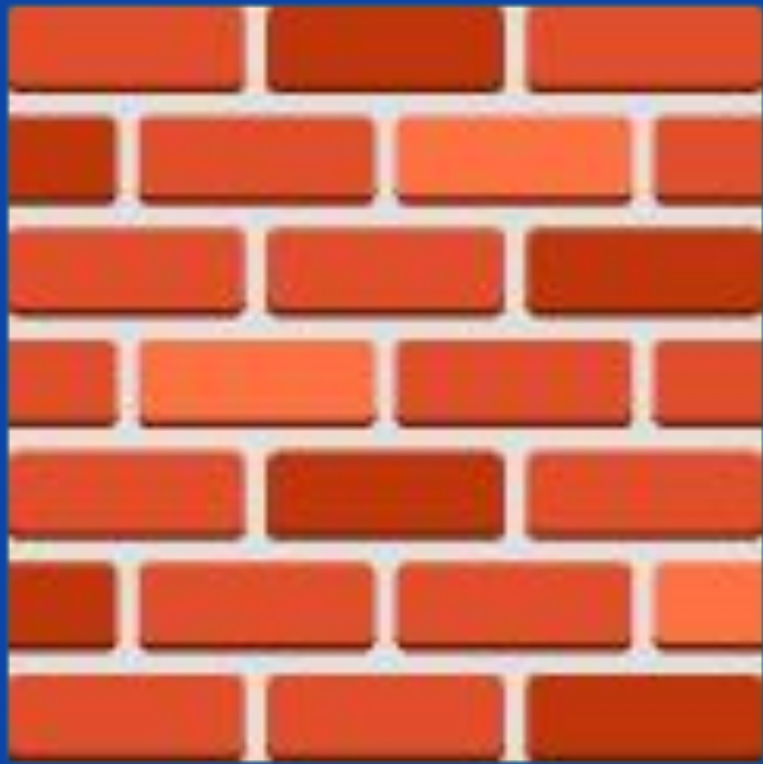
00000000000011a0 <__libc_csu_init>:
11a0:    f3 0f 1e fa    endbr64
11a4:    41 57          push    %r15
11a6:    4c 8d 3d 03 2c 00 00 lea     0x2c03(%rip),%r15    # 3db0 <__frame_dummy_init_array_entry>
11ad:    41 56          push    %r14
11af:    49 89 d6       mov     %rdx,%r14
11b2:    41 55          push    %r13
11b4:    49 89 f5       mov     %rsi,%r13
11b7:    41 54          push    %r12
11b9:    41 89 fc       mov     %edi,%r12d
11bc:    55            push    %rbp
11bd:    48 8d 2d f4 2b 00 00 lea     0x2bf4(%rip),%rbp    # 3db8 <__do_global_ctors_aux_fini_array_entry>
11c4:    53            push    %rbx
11c5:    4c 29 fd       sub     %r15,%rbp
11c8:    48 83 ec 08    sub     $0x8,%rsp
11cc:    e8 2f fe ff ff callq   1000 <_init>
11d1:    48 c1 fd 03    sar     $0x3,%rbp
11d5:    74 1f          je      11f6 <__libc_csu_init+0x56>
11d7:    31 db          xor     %ebx,%ebx
11d9:    0f 1f 80 00 00 00 00 nopl    0x0(%rax)
11e0:    4c 89 f2       mov     %r14,%rdx
11e3:    4c 89 ee       mov     %r13,%rsi
11e6:    44 89 e7       mov     %r12d,%edi
11e9:    41 ff 14 df    callq   *(%r15,%rbx,8)
11ed:    48 83 c3 01    add     $0x1,%rbx
11f1:    48 39 dd       cmp     %rbx,%rbp
11f4:    75 ea          jne     11e0 <__libc_csu_init+0x40>
11f6:    48 83 c4 08    add     $0x8,%rsp
11fa:    5b            pop     %rbx
11fb:    5d            pop     %rbp
11fc:    41 5c          pop     %r12
11fe:    41 5d          pop     %r13
1200:    41 5e          pop     %r14
1202:    41 5f          pop     %r15
1204:    c3            retq
1205:    66 66 2e 0f 1f 84 00 data16  nopw    %cs:0x0(%rax,%rax,1)

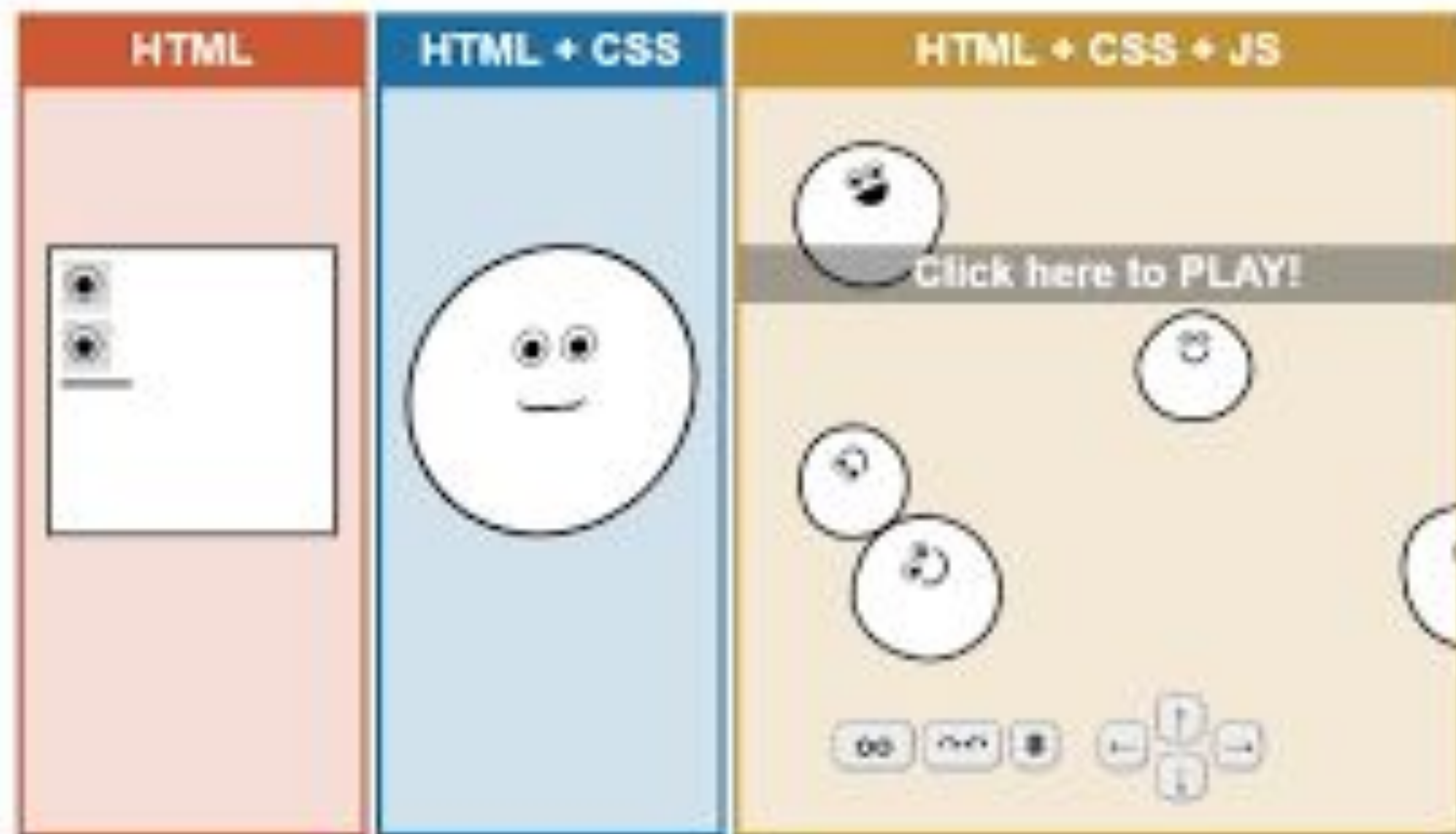
```

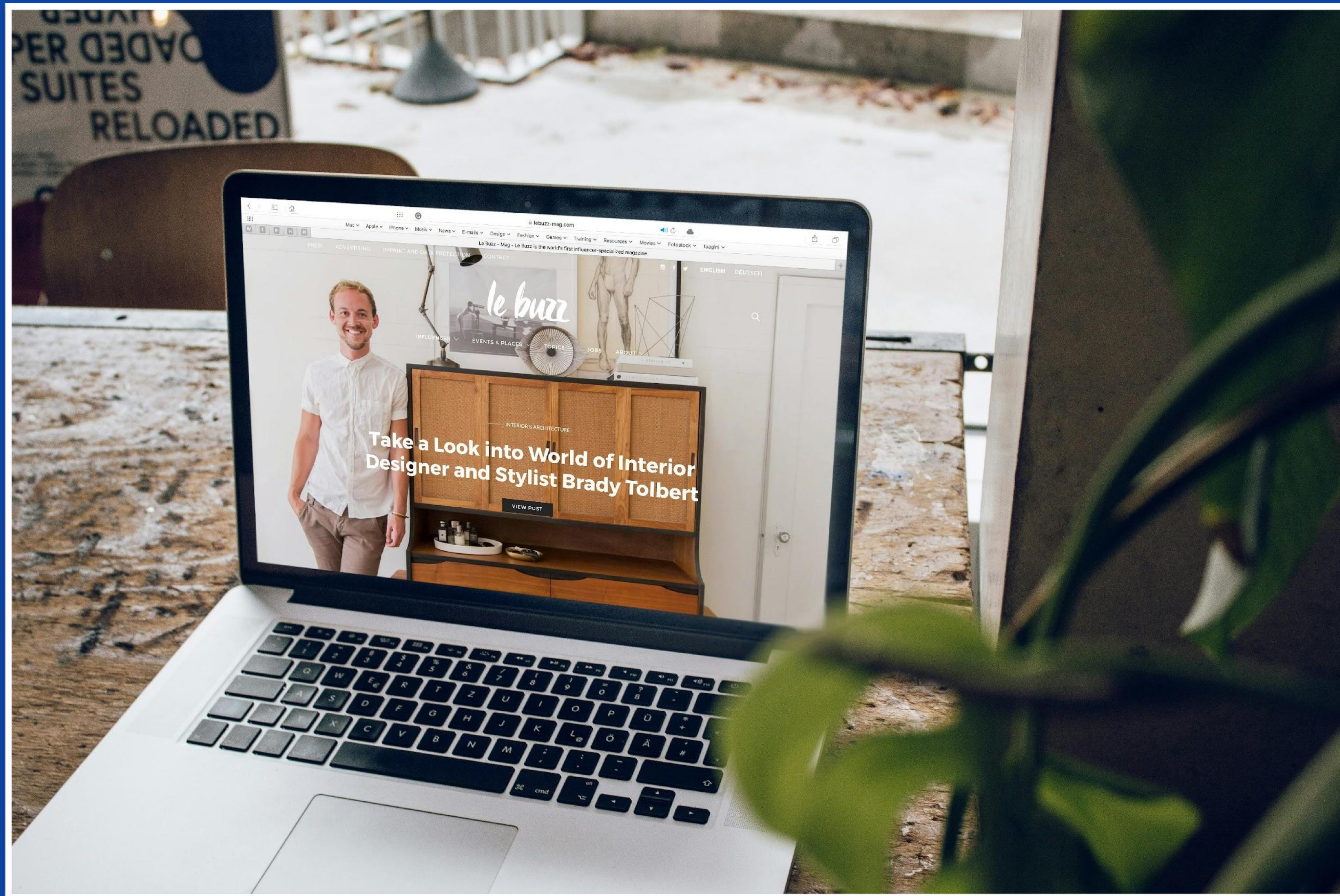
A título de curiosidade, isso é
Assembly.

**Como um site é
desenvolvido?**



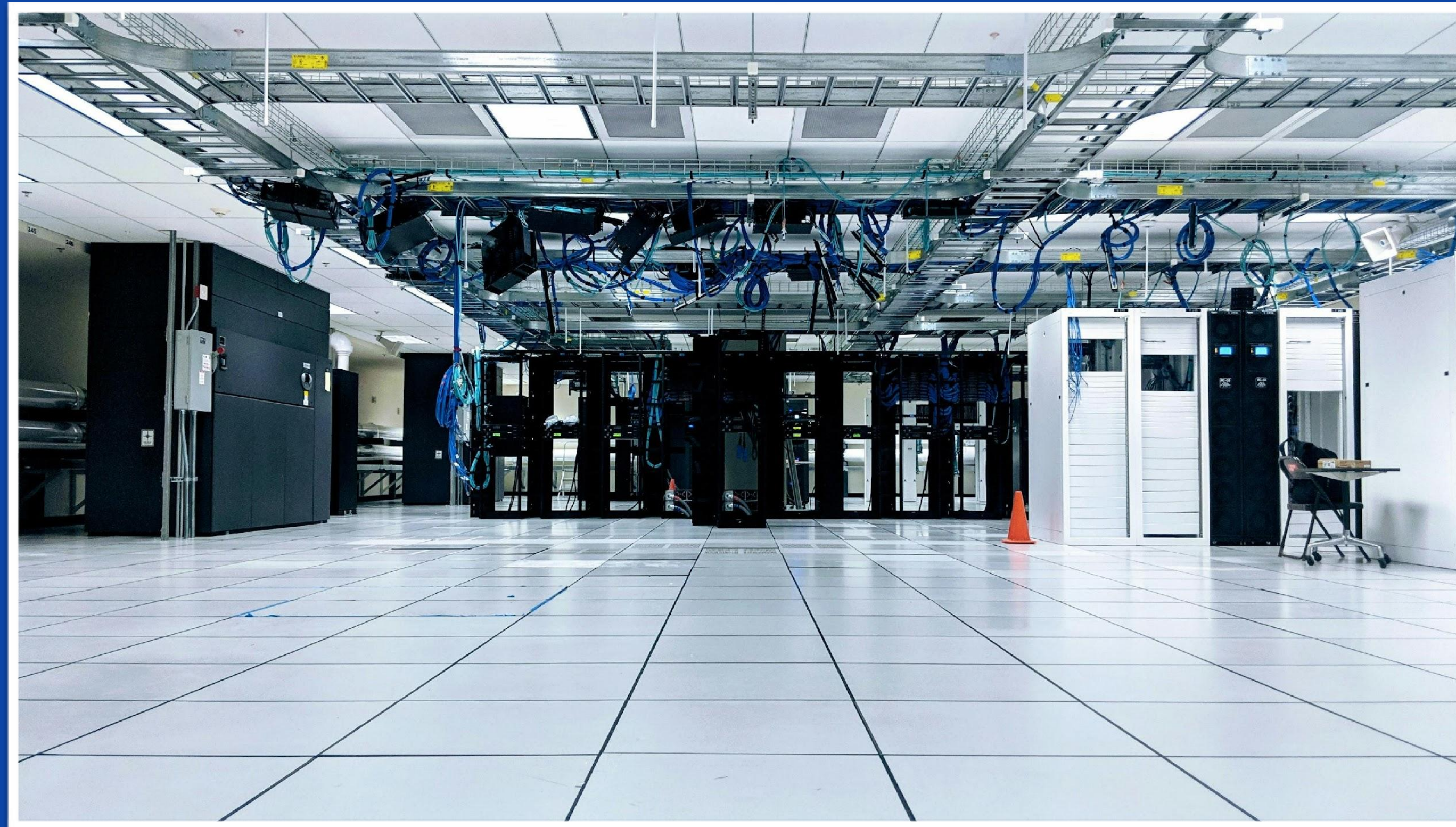




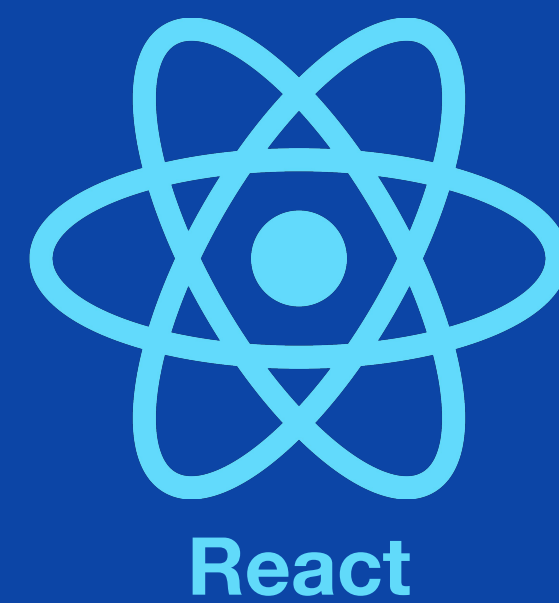
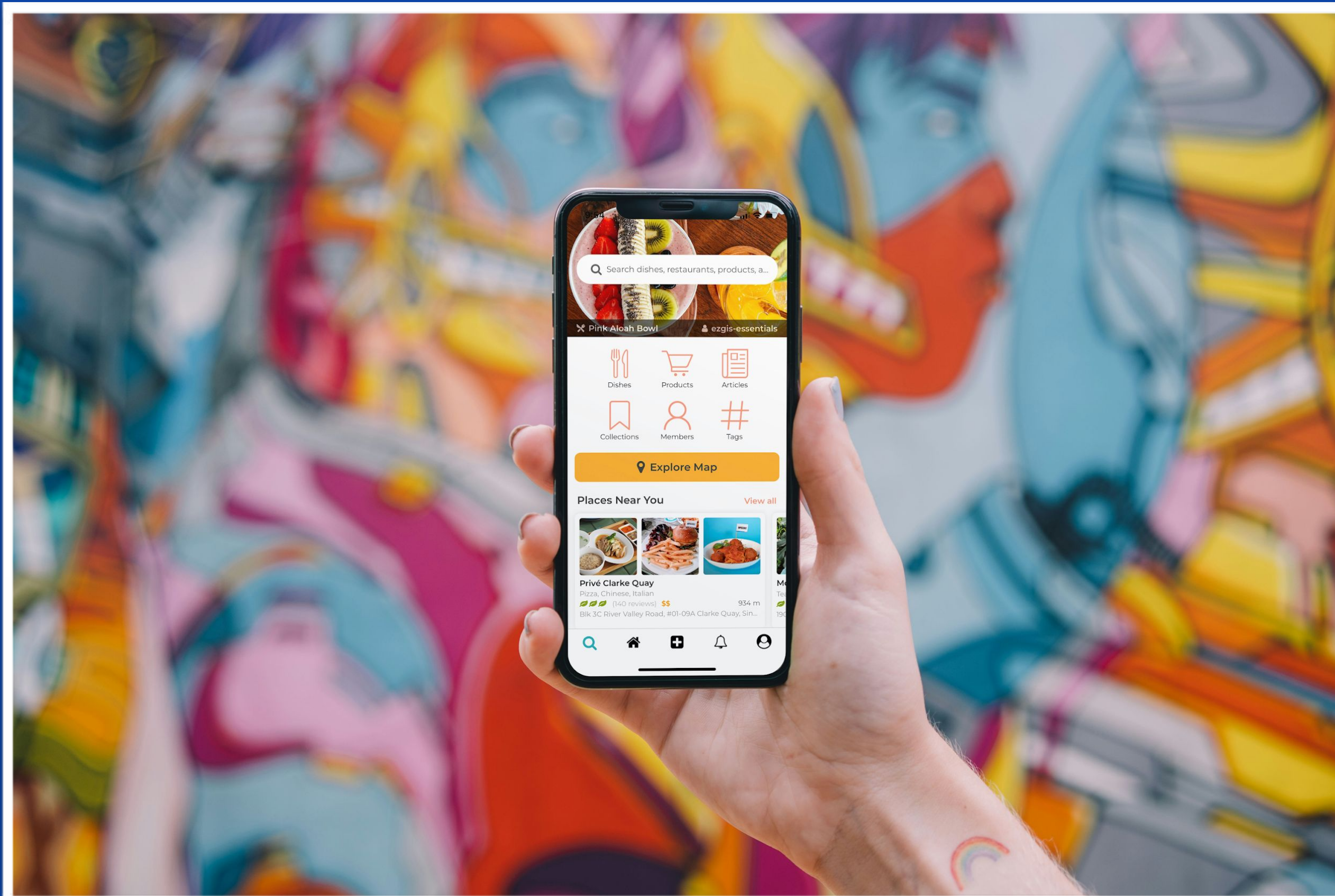


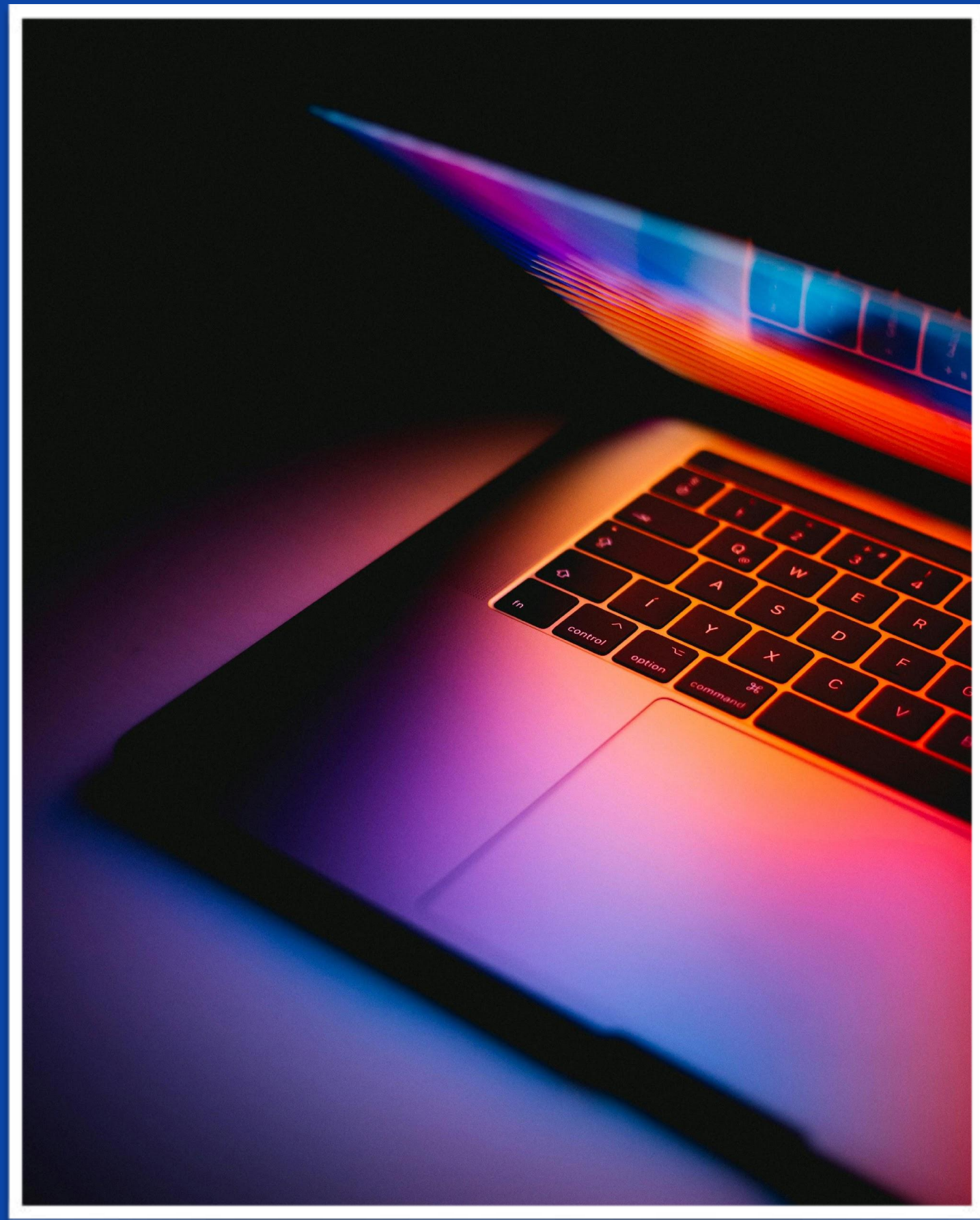


“Tudo que puder ser escrito com JavaScript
eventualmente vai ser escrito com JavaScript”



node JS®





Electron



Permite executar código javascript no servidor, indo além do tradicional contexto do navegador.

Serve como Interpretador do código que vamos escrever

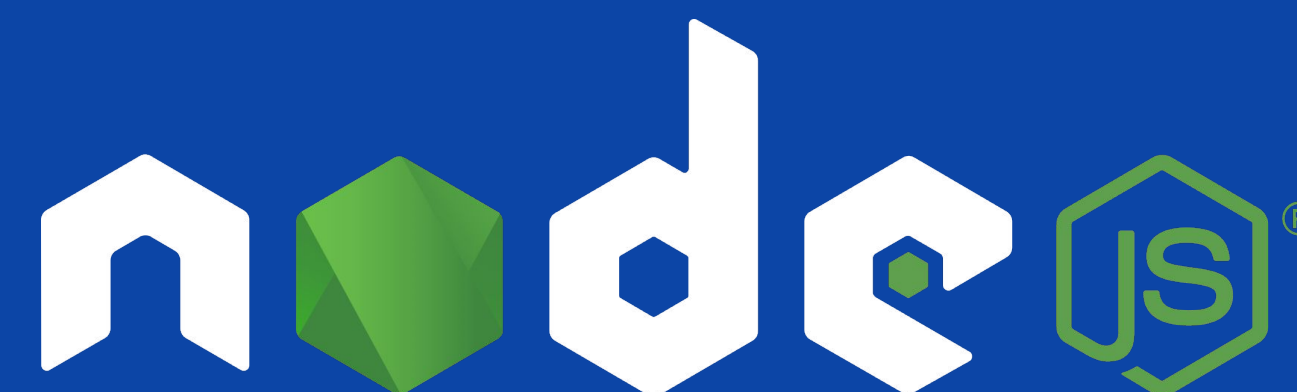
Diversos pacotes pronto para usar, disponíveis através do npm.



Linguagem de Programação

**Desenvolvida originalmente
para ser usada no navegador**

Usada muito em Front End



Ambiente de Execução

**Permite a execução de
JavaScript fora do navegador**

Usada muito em Back End

Como um programa funciona?

Entrada e Saída

Voltando no exemplo da cozinha...





Entrada



Saída



Programa

Programa que calcula a média dos alunos:

Nota1
Nota2
Nota3



Média

Programa que calcula a média dos alunos:



1. $Soma = Nota1 + Nota2 + Nota3$
2. $Resultado = Soma / 3$

**Soma, Nota1, Nota2, Nota3,
Resultado
são todos variáveis**

O que são variáveis?



São elementos fundamentais que permitem armazenar e manipular dados em um programa. Elas representam locais na memória do computador onde valores podem ser armazenados e posteriormente recuperados ou modificados durante a execução do programa.



As variáveis têm nomes que servem como identificadores únicos, permitindo que os programadores se refiram a elas de forma fácil e compreensível. Cada variável possui um tipo de dado associado.

**Como criar uma variável
em JavaScript?**

Três formas principais:

var

**escopo
global**

**valor
pode
ser
alterado**

let

**escopo
local**

**valor
pode
ser
alterado**

const

**escopo
global**

**o valor não
muda, é
constante**

Três formas principais:

```
var nome = 'João'
```

```
let nome = 'João'
```

```
const nome = 'João'
```


Importante!

Muito cuidado com os nomes!!!

Nomes



Nomear variáveis de forma clara e estruturada é essencial para facilitar a leitura e compreensão do código.



Escolha nomes descritivos que indiquem o propósito ou função da variável no contexto do programa.



Evite abreviações ambíguas ou nomes genéricos que não transmitam o significado da variável.



Uma boa prática de nomenclatura de variáveis não apenas facilita a compreensão do código pelo próprio autor, mas também por outros programadores que possam revisar ou dar manutenção ao código.

Convenções



camelCase: começar com a primeira letra minúscula e a primeira letra da segunda palavra maiúscula.



UpperCamelCase: todas as primeiras letras são maiúsculas.



snake_case: utilizamos underscore no lugar dos espaços.

**Agora vamos executar nosso
primeiro código!**

O primeiro passo na programação

A Tradição: Hello World!

 **Texto:** Em quase todas as linguagens de programação, o primeiro programa que escrevemos é o "Hello World" (Olá Mundo).

 **Curiosidade:** Existe uma "lenda" entre programadores de que, se você não começar por esse comando, terá má sorte aprendendo a linguagem!

 **Objetivo:** Fazer o computador exibir uma mensagem para nós usando o comando: `console.log(" ")`

Atividade

Instruções:

1. No VS Code, abra o arquivo que você criou (ou crie um chamado teste.js).
2. Escreva o comando para exibir a frase: "Estou programando em JavaScript!".
3. Salve o arquivo.
4. No terminal, execute usando o Node: `node teste.js`.

Desafio Extra: Tente imprimir o resultado de uma conta matemática, como `console.log(10 + 5);`. O que acontece?

**Hoje, vamos trabalhar
com um tipo de dado:
String**

**Alguém sabe o que são
Strings?**

String

É um dos tipos fundamentais de dados.

Traduzindo do inglês, é uma teia, no sentido de teia ou cadeia de caracteres.

É usada para armazenar texto.

Podem ser criadas usando áspas simples (' ') ou dúplas ("").

String

```
let nome = 'João'
```

```
let mensagem = "Contem comigo!"
```


String

Assim como vimos ontem, quando utilizamos um tipo de aspas, se necessário devemos usar o outro tipo de aspas para escapar o código.

```
let exemplo = 'Isso é uma "string" com aspas duplas dentro dela.'
```

```
let exemplo = "Isso é uma 'string' com aspas simples dentro dela."
```


Concatenação de Strings

É o processo de combinar duas ou mais strings para criar uma nova string. Uma das formas de concatenar strings é utilizando o operador de adição (+), que une os valores das strings.

```
let nome = "João"
```

```
let saudacao = "Olá " + nome + "!"
```

```
let fraseCompleta = saudacao + "Hoje é um belo dia."
```

Hora da Prática!

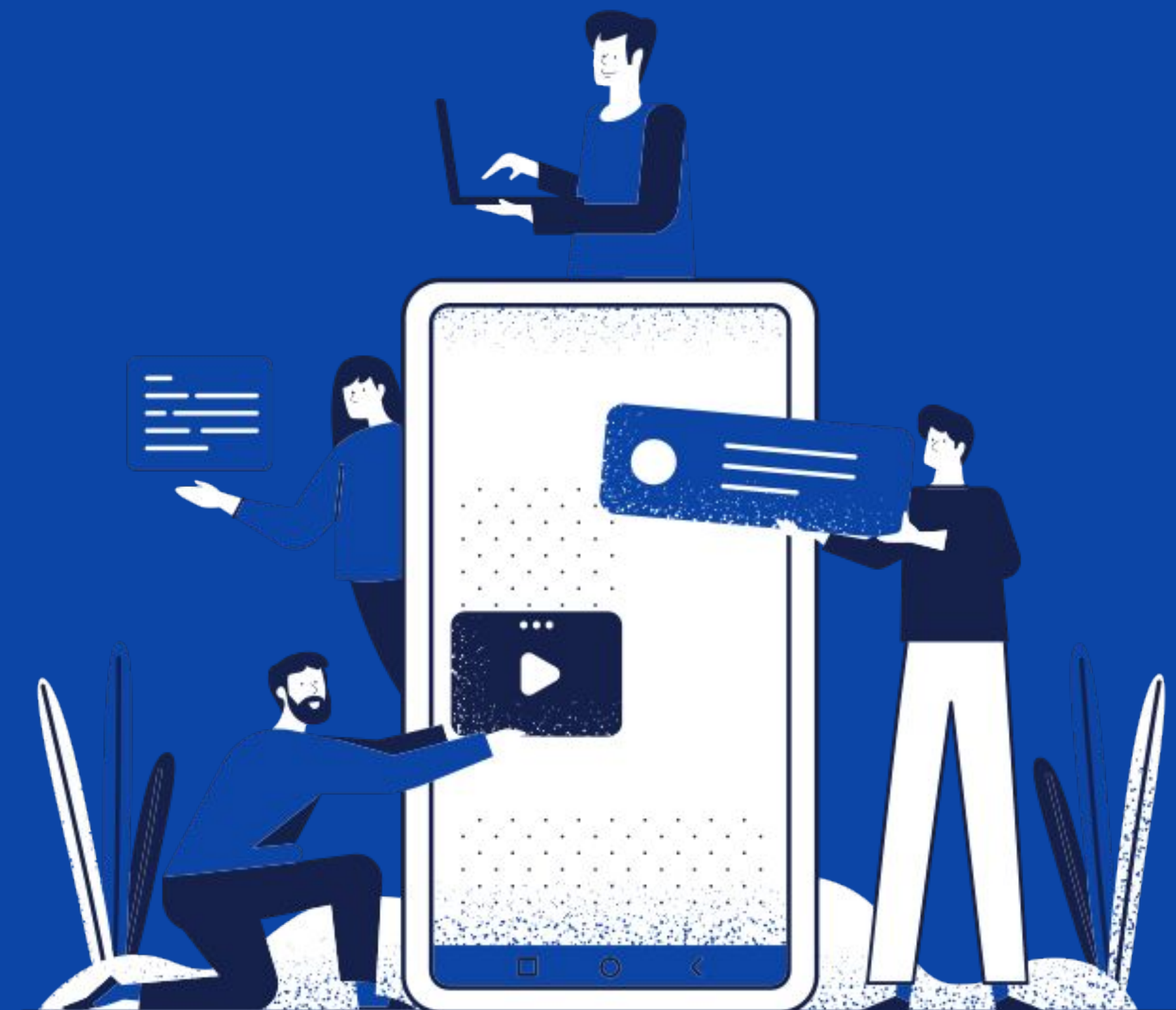
Atividade

1. Abrir o Terminal.
2. Navegar até a Pasta Módulo 1 dentro de Documentos.
3. Criar uma pasta chamada variaveis, e abrir o VS Code dentro dela.
4. Criar um arquivo saudacao.js. Dessa vez, usando o editor de texto do VS Code mesmo, criar uma variável chamada nome, com o seu nome.
5. Criar uma variável chamada mensagem com uma mensagem de olá.
6. Usar a concatenação de strings para unir a mensagem com o nome no formato: olá, nome.
7. Usar o comando `console.log()` para imprimir a variável.
8. Executar o programa usando o Node.js.

JS saudacao.js U X

Modulo02 > atividades > string > JS saudacao.js > ...

```
1 // declarar uma variável chamada "nome" e atribuir o seu próprio nome a ela (como texto)
2 // declarar uma variável chamada "mensagem" e atribuir uma saudação a ela (ex: "Olá")
3 // criar uma terceira variável para unir (concatenar) a mensagem, um espaço e o nome
4 // usar o comando console.log() para imprimir a variável com a mensagem final
```

Interação com o Usuário



Entrada



Saída

Na atividade, usamos coisas que já estavam ali, nas variáveis que criamos. Como podemos interagir com o usuário do programa, para que ele dê uma entrada, e o programa dê uma saída?

Interatividade (Entrada de Dados)

O que é: Até agora o código era fixo. Agora, vamos fazer o programa "ouvir" o usuário.

A Ferramenta: Usaremos o pacote prompt-sync.

Instalação (No Terminal):

```
• nicolasmotta@nicolasmotta:~/CursoLionsDev$ npm install prompt-sync  
added 3 packages in 1s
```


Para usar, precisamos de dois passos simples:

```
//Exemplo de uso:  
// 1. Prepara a ferramenta  
const prompt = require('prompt-sync')();  
  
// 2. Pergunta e guarda a resposta (sempre vira String!)  
let nome = prompt("Qual seu nome? ");  
let cidade = prompt("Onde você mora? ");  
  
console.log("Olá " + nome + ", de " + cidade + "!");
```


Atividade

1. Inicialize o projeto usando: `npm init - y`
2. Instale o `prompt-sync` na sua pasta.
3. Crie um arquivo `.js`
4. Peça ao usuário: Nome do pet e Idade do pet.
5. Exiba: "O [NOME] tem [IDADE] anos!".
6. Execute com o comando: `node`

JS prompt-sync.js X

Modulo02 > atividades > JS prompt-sync.js > ...

```
1 // importar a biblioteca prompt-sync para receber entradas
2 // declarar variavel para armazenar o nome do pet usando prompt
3 // declarar variavel para armazenar a idade do pet usando prompt
4 // imprimir a mensagem final concatenando o nome e a idade
```

Além do prompt-sync, temos a forma nativa da linguagem de interagir com o usuário...

Stdin e Stdout

Stdin é uma referência ao fluxo de entrada padrão do programa, geralmente associado à entrada de dados pelo teclado.

Em português: é a entrada

Stdout é uma referência ao fluxo de saída padrão do programa, geralmente as-sociado ao console onde as mensagens são exibidas.

Em português: é a saída

Stdin

Imagine que você está em um teclado digitando uma mensagem.

O que você digita é como a entrada, ou seja, o que você está enviando para o computador.

O stdin é como o ouvido do computador para ouvir o que você está digitando.

Ele espera pacientemente até você pressionar Enter para enviar sua mensagem.

Quando isso acontece, o stdin pega o que você escreveu e diz ao computador o que você digitou.

Stdin

```
// Definindo uma variável para armazenar a entrada do usuário
let entradaUsuario = '';

// Escutando o evento 'data' para receber dados de entrada do usuário
process.stdin.on('data', function (data) {
    // Convertendo os dados de entrada (Buffer) para string
    entradaUsuario = data.toString();

    // Exibindo a entrada do usuário no console
    console.log('Você digitou:', entradaUsuario);
});
```

Stdout

Temos duas principais formas de saída, uma vocês já usaram...

```
process.stdout.write(“qualquer texto! \n”)
```

```
console.log(‘Qualquer string’);
```

Em seguida, para encerrar o processo usamos:

```
process.exit();
```


Exemplo com Entrada e Saída

```
console.log('Digite algo e pressione Enter: ');  
  
process.stdin.on('data', function(data) {  
    let entrada = data.toString();  
  
    process.stdout.write('Olá, mundo!\n' + entrada);  
  
    process.exit();  
});
```


Atividade

1. Na pasta variáveis, crie um novo arquivo chamado boas-vindas.js.
2. Crie uma variável chamada nome, sem atribuir um nome.
3. Use o `console.log("")` para perguntar o nome do usuário.
4. Usando o `stdin`, colete o nome do usuário.
5. Crie uma variável chamada saudação personalizada no formato "Olá, Nome".
6. Use o `console.log()` para imprimir essa saudação personalizada no console.
7. Execute o programa usando o Node.js.

```
JS stdin_stdout.js U X
Modulo02 > atividades > entrada_dados > JS stdin_stdout.js
1 // declarar uma variável para o nome, sem atribuir nenhum valor inicial a ela
2 // usar o console.log() para imprimir a pergunta no terminal (ex: "Qual é o seu nome?")
3
4 // a estrutura process.stdin.on(...) já foi fornecida para capturar a digitação
5 // (dentro do bloco): converter a entrada (data) para texto usando .toString() e salvar na variável nome
6 // (dentro do bloco): declarar uma nova variável para a saudação, unindo o texto "Olá, " com a variável nome
7 // (dentro do bloco): usar o console.log() para imprimir a variável de saudação final
```

Revisão



Entrada e Saída

Prompt-Sync
Stdin e Stdout



Variáveis

const, let, var



Strings

"" , ' ' , ``

Esclarecendo a aula passada...

```
import promptSync from "prompt-sync"

const prompt = promptSync();

const nome = prompt("Digite o nome do pet: ");
const idade = prompt("Digite a idade do pet: ");
console.log(`O ${nome} tem ${idade} anos!`);
```

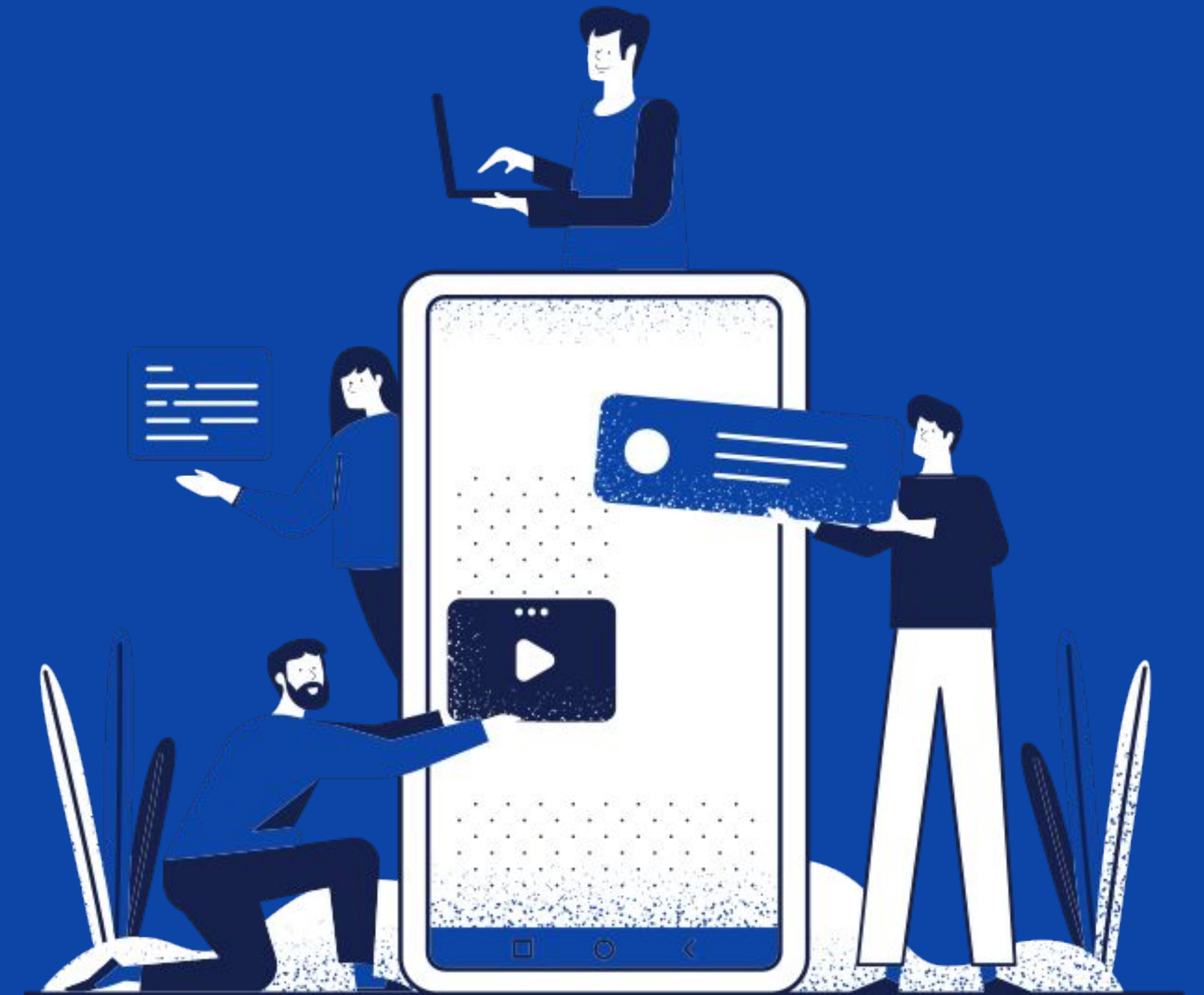
```
let nome
let idade

console.log('Como é seu nome?')

process.stdin.once('data', function(data) {
  nome = data.toString().trim()
  console.log('Qual é sua idade?')

  process.stdin.once('data', function(data) {
    idade = parseInt(data.toString().trim())
    processamento(nome, idade)
    process.exit()
  })
})

function processamento(nome, idade) {
  console.log(`Olá ${nome}, você tem ${idade} anos`)
}
```

Fundamentos de Programação e Introdução ao Debugging com Node.js

Concatenação e Interpolação de Strings



Concatenação

```
let nome = "João"
```

```
let mensagem = "Olá, " + nome
```


Interpolação

```
let nome = "João"
```

```
let mensagem = `Olá, ${nome}`
```


Tipos de Dados



Boolean

Verdadeiro (true), ou Falso (false).



Null

Ausência de dados.



Number

Números, podendo ser inteiros (int), reais (float), NaN, infinito (infinity).



Undefined

O valor não foi declarado ainda.



String

Uma teia de caracteres, em outras palavras, texto.



Object

Uma coleção de propriedades, onde propriedades são associações de nomes e valores.



Boolean

```
let portaAberta = false
```



Null

```
let vazio = null
```



Number

```
let idade = 15
```



Undefined

```
let aindaNaoSei
```



String

```
let nome = 'João'
```



Object

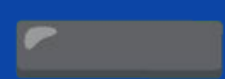
```
let carro = new Object()  
carro.marca = 'Ford'  
carro.ano = 2024
```


Operadores



Adição

Igual a adição em matemática.



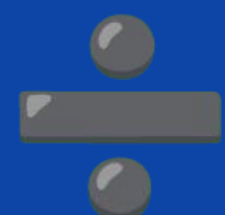
Subtração

Igual a subtração em matemática.



Multiplicação

Igual a multiplicação em matemática.



Divisão

Igual a divisão em matemática.



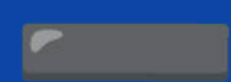
Módulo

O resto da divisão.



Adição

```
let soma = 10 + 5 // Resultado: 15
```



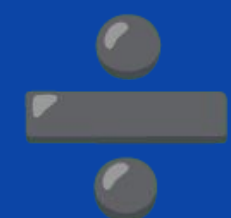
Subtração

```
let diferenca = 10 - 5 // Resultado: 5
```



Multiplicação

```
let produto = 10 * 5 // Resultado: 50
```



Divisão

```
let quociente = 10 / 5 // Resultado: 2
```



Módulo

```
let resto = 10 % 3 // Resultado 1
```

Atividade:

Usando a entrada e saída, desenvolva um programa que solicite ao usuário duas notas, e calcule a média entre elas.

Cuidado: não se esqueça de especificar que a nota é do tipo ponto flutuante (float) na entrada. Para isso, usaremos o `parseFloat()`, assim recebendo o dado com o tipo correto.

```
media_notas.js M X
Modulo02 > atividades > media_notas.js > ...
1 // importar a biblioteca prompt-sync para receber entradas
2 // declarar variável para a nota 1, convertendo para float com parseFloat()
3 // declarar variável para a nota 2, convertendo para float com parseFloat()
4 // criar uma variável para calcular e armazenar a média aritmética
5 // imprimir a mensagem final exibindo o resultado da média
```


Revisão



Concatenação e Interpolação

```
let nome = "João"
```

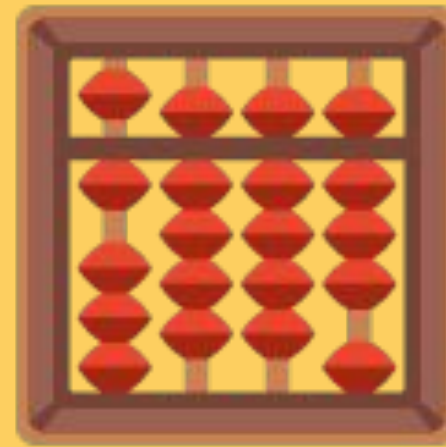
```
let mensagemConcatenada = "Olá, " + nome + "!"
```

```
let mensagemInterpolada = `Olá ${nome}!`
```



Tipos de Dados

```
let portaAberta = false  
let vazio = null  
let idade = 15  
let media = 7.5  
let aindaNaoSei  
let nome = 'João'
```



Operadores Matemáticos

```
let soma = 10 + 5
```

```
let diferenca = 10 - 5
```

```
let produto = 10 * 5
```

```
let quociente = 10 / 5
```

```
let resto = 10 % 3
```


**Mais alguns operadores bem
úteis...**

Condicionais

If

Se (alguma condição), então {uma ação}

If ... Else

Se (alguma condição), então {uma ação}

Senão {uma ação geral}

If ... Else If ... Else

Se (alguma condição), então {uma ação}

Senão Se (alguma outra condição), então {uma outra ação}

Senão {uma ação geral}

Operadores Lógicos

Operadores Lógicos: And (E)

```
let portaFechada = true  
let janelaFechada = true  
let podeLigarAC = portaFechada && janelaFechada  
  
// podeLigarAC = true
```


Operadores Lógicos: And (E)

```
let portaFechada = false  
let janelaFechada = true  
let podeLigarAC = portaFechada && janelaFechada  
  
// podeLigarAC = false
```

Operadores Lógicos: OR (OU)

```
let estaFrio = false
let estaAbafadoDentroDaSala = true
let podeLigarAC = estaFrio || estaAbafadoDentroDaSala

// podeLigarAC = true
```

Operadores Lógicos: OR (OU)

```
let estaFrio = true
let estaAbafadoDentroDaSala = true
let podeLigarAC = estaFrio || estaAbafadoDentroDaSala

// podeLigarAC = true
```


Operadores Lógicos: OR (OU)

```
let estaFrio = false
let estaAbafadoDentroDaSala = false
let podeLigarAC = estaFrio || estaAbafadoDentroDaSala

// podeLigarAC = false
```

Operadores Lógicos: NOT (negação)

```
let estaFrio = false
```

```
let estaAbafadoDentroDaSala = false
```

```
let podeLigarAC = estaFrio || estaAbafadoDentroDaSala
```

```
// podeLigarAC = false
```

```
let naoPodeLigarAC =! podeLigarAC
```

```
// naoPodeLigarAC = true
```


&& **And**
Conjunção Lógica - E. Retorna verdadeiro se ambos tiverem o mesmo valor.

|| **Or**
Disjunção Lógica - Ou. Retorna verdadeiro se um dos valores for verdadeiro.

! **Not**
Negação Lógica - Não. Retorna o oposto, verdadeiro se for falso, falso se for verdadeiro.

== Igualdade de Valor

```
let x = 5
x == 5    // Verdadeiro
x == '5'  // Verdadeiro
```

=== Igualdade de Valor e Tipo

```
let x = 5
x === 5   // Verdadeiro
x === '5' // Falso
```

!= Diferença de Valor

```
let x = 5
x != 6   // Verdadeiro
x != 5   // Falso
```

!== Diferença de Valor e Tipo

```
let x = 5
x !== 6   // Verdadeiro
x !== 5   // Falso
```

 Não confundir com Atribuição (=)

> Maior

```
let x = 5
x > 5    // Falso
x > 3    // Verdadeiro
```

< Menor

```
let x = 5
x < 5    // Falso
x < 6    // Verdadeiro
```

>= Maior ou Igual

```
let x = 5
x >= 3    // Verdadeiro
x >= 5    // Verdadeiro
```

<= Menor ou Igual

```
let x = 5
x <= 5    // Verdadeiro
x <= 3    // Falso
```


Podemos juntar expressões lógicas...

Atividade 1:

Desenvolva um programa simples que pede para o usuário digitar um número e diz se ele é ímpar, ou não.

Dica: use o operador de modulo (%), e não se esqueça de tratar do 0.

```
JS par_impar.js M X
Modulo02 > atividades > JS par_impar.js > ...
1 // importar a biblioteca prompt-sync
2 // inicializar o prompt
3 // declarar uma variável para receber o número digitado pelo usuário
4 // (lembre-se de converter a entrada de texto para número)
5 // criar uma primeira condição (if) para verificar se o número é exatamente igual a 0
6 // se for, imprimir uma mensagem informando que o número é 0
7 // criar uma segunda condição (else if) para verificar se o resto da divisão do número por 2 é diferente de 0
8 // se a condição for verdadeira, imprimir uma mensagem dizendo que o número é ímpar
9 // criar a condição final (else) para os números restantes
10 // imprimir uma mensagem informando que o número é par
```


Atividade 2:

Desenvolva um programa simples que pergunta se o usuário gosta de café. Use uma condicional para salvar a entrada dele numa variável booleana `gostaDeCafe`, e baseado nela, crie outra condicional, para imprimir uma mensagem personalizada caso ele goste, e caso ele não goste.

JS café.js M X

Modulo02 > atividades > JS café.js > ...

```
1 // importar a biblioteca prompt-sync para receber entradas
2 // usar o prompt para perguntar se o usuário gosta de café e salvar a resposta em uma variável de texto
3 // declarar a variável booleana gostaDeCafe (usando let)
4 // criar a primeira condicional (if/else) para checar a resposta de texto digitada pelo usuário
5 // dentro dessa condicional, atribuir o valor true ou false para a variável gostaDeCafe
6 // criar uma segunda condicional (if/else) que avalie apenas o valor armazenado na variável booleana gostaDeCafe
7 // imprimir a mensagem personalizada caso o valor seja verdadeiro (true)
8 // imprimir uma mensagem diferente caso o valor seja falso (false)
```


Atividade 3:

Agora, desenvolva um programa que solicite o nome do usuário, e sua idade, com esses dados, ele imprima uma mensagem dizendo em qual ano ele nasceu. Para facilitar, crie uma constante com o ano atual, aí não é preciso pedir mais uma entrada.

Cuidado: A idade deve ser calculada como inteiro. Outra coisa, na hora de imprimir a mensagem, fale sobre as duas possibilidades - caso já fez aniversário (idade) ou não (idade - 1).

```
JS calculo_idade.js X
Modulo02 > atividades > JS calculo_idade.js > ...
1 // importar a biblioteca prompt-sync para receber entradas
2 // criar uma constante para armazenar o ano atual
3 // declarar variável para armazenar o nome do usuário usando prompt
4 // declarar variável para a idade, convertendo para inteiro com parseInt()
5 // calcular o ano de nascimento (anoAtual - idade) para quem já fez aniversário
6 // calcular o ano de nascimento (anoAtual - idade - 1) para quem não fez aniversário
7 // imprimir a mensagem final com o nome e as duas opções de ano de nascimento
```

Hora da Prática

Atividade:

Desenvolva um programa que pergunta o nome e a idade do usuário.

Caso ele seja maior de idade, imprima a mensagem: “Nome, você já é maior de idade.”.

Caso ele seja de menor, imprima a mensagem “Nome, você vai ser maior de idade em ANOS anos.” (calcular em quantos anos ele será maior de idade).

JS maior_idade.js M X

Modulo02 > atividades > JS maior_idade.js > ...

```
1 // importar a biblioteca prompt-sync para receber entradas
2 // declarar uma variável para o nome, recebendo a entrada do usuário
3 // declarar uma variável para a idade, convertendo a entrada para número inteiro com parseInt()
4 // criar uma condicional (if/else) para verificar se a idade digitada é maior ou igual a 18
5 // dentro da condição verdadeira (se for maior de idade), imprimir a mensagem informando que ele já é maior
6 // dentro da condição falsa (else), criar uma variável para calcular a diferença (18 menos a idade do usuário)
7 // ainda dentro do bloco falso, imprimir a mensagem informando quantos anos faltam usando a variável calculada
```


Arrays

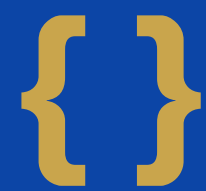
Arrays



É uma estrutura de dados que permite armazenar múltiplos valores em uma única variável.



Os elementos de um array podem ser de diferentes tipos de dados, incluindo números, strings, booleanos, objetos e até mesmo outros arrays.



Tem funcionamento semelhante aos conjuntos da matemática.



Os elementos podem ser acessados através de índices, começando no 0.

Para acessar um elemento:

```
let exemplo = [10, 20, 30, 40, 50]
let umaVariavelQualquer = exemplo[1]
console.log(exemplo[3])
console.log(`0 primeiro elemento do array é ${exemplo[0]}.`)
```

Para acessar um elemento:

```
let exemplo = [10, 20, 30, 40, 50]  
exemplo[1]
```


Manipulação de Arrays

Push (Adicionar Elemento)

```
let exemplo = [10, 20, 30, 40, 50]  
exemplo.push(60)  
console.log(exemplo)  
// Vai imprimir [ 10, 20, 30, 40, 50, 60 ]
```


Pop (Remover Último Elemento)

```
let personagensPrincipais = ['Shrek', 'Fiona',  
    'Príncipe Encantado']  
personagensPrincipais.pop()  
console.log(personagensPrincipais)  
// Vai imprimir [ 'Shrek', 'Fiona' ]
```

Shift (Remover Primeiro Elemento)

```
let personagensPrincipais = ['Shrek', 'Fiona']  
personagensPrincipais.shift()  
console.log(personagensPrincipais)  
// Vai imprimir [ 'Fiona' ]
```

Splice (Remover Elemento Específico)

```
let personagensPrincipais = ['Shrek', 'Fiona', 'Principe  
Encantado']  
personagensPrincipais.splice(0, 2)  
console.log(personagensPrincipais)  
// Vai imprimir [ 'Principe Encantado' ]
```


Splice (Remover Elemento Específico)

```
let numeros = [10, 20, 30, 40, 50]  
personagensPrincipais.splice(2, 1)  
console.log(numeros)  
// Vai imprimir [ 10, 20, 40, 50 ]
```

Unshift (Adicionar Elemento no Início)

```
let personagensPrincipais = ['Fiona']  
personagensPrincipais.unshift('Principe Encantado')  
console.log(personagensPrincipais)  
// Vai imprimir [ 'Principe Encantado', 'Fiona' ]
```


Includes (Verificar se faz Parte)

```
let personagensPrincipais = ['Principe Encantado', 'Fiona']  
if (personagensPrincipais.includes('Principe Encantado')) {  
    console.log('O filme é Shrek 2 ou 3')  
}  
// .includes('') retorna um valor booleano
```

IndexOf (Retorna a Posição)

```
let personagensPrincipais = ['Principe Encantado', 'Fiona']  
console.log(personagensPrincipais.indexOf('Principe Encantado'))  
// Vai imprimir 0
```

Length (Retorna o Tamanho do Array)

```
let personagensPrincipais = ['Principe Encantado', 'Fiona']  
console.log(personagensPrincipais.length)  
// Vai imprimir 2
```


Hora da Prática

Atividade 1:

Vamos praticar a criação e funcionamento de Arrays com uma dinâmica entre vocês... Para facilitar, essa atividade não precisa pedir entrada do usuário.

Crie um array chamado `minhaMesa`, dentro dele, coloque strings com o nome dos alunos que sentam na mesma mesa que você - incluindo o seu. Depois imprima uma mensagem no console do tipo:

“Meu nome é NOME, meus colegas são COLEGA, COLEGA, COLEGA.”

Para criar esta mensagem, use interpolação de string, e coloque o seu nome/nome dos colegas, acessando o nome de cada um pela posição no array, incluindo você, em ordem da esquerda pra direita.

Atividade 2:

Agora vamos desenvolver um programa que peça ao usuário que digite 2 notas (uma de cada vez) referente a duas provas, sendo elas, prova1 e prova2 e armazene as notas em um array, em seguida, calcule a media das notas, e finalmente imprima uma mensagem com o resultado da média.

Dicas: crie o array notas vazio, crie variáveis prova1 e prova2 para salvar a entrada do usuário, não se esqueça de usar parseFloat(). Use o push() para adicionar o valor de prova1 e prova2 ao array notas, crie uma variável com o cálculo da média, e calcule referenciando as posições do array.

Atividade 3:

Vamos criar um pequeno gerenciador de lista de tarefas! Crie um programa que inicie com um array vazio chamado `listaTarefas`. Peça ao usuário para digitar 3 tarefas que ele precisa fazer no dia (uma de cada vez) e armazene-as no array. Em seguida, imprima no console a quantidade de tarefas pendentes usando uma mensagem como: "Você tem X tarefas na sua lista."

Depois, simule que o usuário concluiu a última tarefa do dia: remova o último item do array e imprima a lista atualizada no console para ele ver o que restou.

Dicas: use o método `push()` para adicionar as tarefas da entrada do usuário. Para descobrir a quantidade de itens, use a propriedade `.length` (ex: `listaTarefas.length`). Para remover o último item, utilize o método `pop()`. Se quiser imprimir a lista final de um jeito mais legível, experimente usar o `console.log()` direto no array ou pesquise sobre o método `join()`.

Objetos

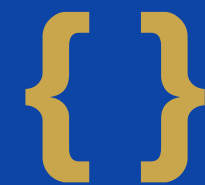
Objetos



É uma estrutura de dados que permite armazenar e organizar informações relacionadas.



As propriedades de um objeto podem conter diferentes tipos.



É composto por um conjunto de propriedades, onde cada propriedade possui um nome (chave) e um valor associado.



As propriedades de um objeto podem ser acessadas e manipuladas usando a notação de ponto (objeto.propriedade)



Cuidado...

É parecido com Array, mas tem funcionalidade diferente. O array tem uma listagem numerada, já o objeto tem uma listagem nomeada.

Criando um Objeto

```
let carro = {  
  fabricante: "Ford",  
  modelo: "Ka",  
  ano: 2009  
}
```

Criando um Objeto

```
let pessoa = {  
  nome: "João Pedro dos Santos",  
  idade: 20,  
  cidadeDeNascimento: "Ponta Grossa"  
}
```

Acessando um Valor do Objeto

```
let carro = {  
  fabricante: "Ford",  
  modelo: "Ka",  
  ano: 2009  
}
```

```
console.log(carro.ano)  
// Vai imprimir 2009
```


Acessando um Valor do Objeto

```
let turma = {  
  totalAlunos: 55,  
  horário: manhã  
}
```

```
turma.totalAlunos = turma.totalAlunos - 5
```

```
console.log(turma.totalAlunos)  
// Vai imprimir 50
```

Objetos podem ter métodos...

```
let curso = {};  
  
curso = {  
  nomeCurso: 'JavaScript',  
  local: 'Lions Startups',  
  ano: 2024,  
  professores: ['Edson', 'Eduardo', 'Jhonatan', 'João'],  
  materia: {  
    modulo1: 'introdução',  
    modulo2: 'variáveis',  
    modulo3: 'operadores',  
  },  
  verificaProfessor: function (professor) {  
    if(this.professores.includes(professor) == true) {  
      console.log('O professor ' + professor + ' professor leciona nessa instituição');  
    } else {  
      console.log('O professor ' + professor + ' não leciona nessa instituição');  
    }  
  }  
}
```

Atividade:

Para encerrar a aula de hoje, vamos praticar a criação e funcionamento de Objetos.

Para facilitar, esse programa não precisa pedir entrada do usuário.

Vamos criar um objeto para guardar as informações da sua pintura favorita. Se ainda não tiver uma, pesquise no google, e escolha.

Esse objeto deve salvar o nome da pintura, o nome do pintor, o ano que ela foi feita e uma frase curta com seu significado, então ele deve ter as seguintes propriedades: nomePintura, nomeArtista, ano, significado.

Em seguida, use o `console.log()` para imprimir uma mensagem do tipo:

“Minha pintura favorita é ... feita por ... em ... e significa ...”.

Laços de Repetição



É uma estrutura que permite executar mais de uma vez o mesmo comando ou conjunto de comandos.



Faz um bloco de código se repetir por um número definido ou indefinido de vezes (finito), enquanto uma condição é satisfeita.



Principais laços: `while`, `do while`, `forEach` e `for`



O laço `for` é amplamente utilizado para executar tarefas repetitivas, como percorrer elementos de um array, processar dados em uma matriz multidimensional, iterar sobre valores em um objeto ou realizar operações matemáticas sequenciais.

```
for (inicialização, condição, atualização) {  
    // código aqui  
}
```



```
for (let i = 0; i <= 5; i += 1) {  
    console.log(i)  
}
```



```
while (condição) {  
    // código aqui  
}
```



```
do {  
    // código aqui  
} while (condição)
```



```
let horas = [11, 12, 13]
```

```
horas.forEach((element) => console.log(element))
```

Hora da Prática

Atividade 1:

Vamos praticar o funcionamento dos laços de repetição. Desenvolva um algoritmo onde através do console, o usuário escolha uma tabuada, e apresente o cálculo da tabuada de forma formatada no console.

Use alguma forma de entrada de dados para perguntar ao usuário qual tabuada ele deseja saber (salve o número numa variável) , e calcule usando o laço de repetição for.

Dica: a condição do for vai ser de 1 ou 0 até 10, e você pode fazer a conta usando tabuada multiplicado por i.

Desafio: faça com que o programa calcule até 100.

Atividade 2:

Crie um array chamado `listaDeCompras` contendo quatro itens: "Arroz", "Feijão", "Macarrão" e "Carne". Utilize um laço `for` que vá de 0 até o tamanho do array (usando `.length`) e imprima cada um dos itens no console no formato: "Item: [nome do item]".

Atividade 3:

Crie uma variável temperaturaAgua começando com o valor 90. Usando um laço while, crie uma condição para que o bloco se repita enquanto a temperatura for menor que 100. Dentro do laço, imprima "A temperatura está em [valor] graus. Aquecendo..." e aumente o valor da temperatura em 2 a cada repetição.

Atividade 4:

Um evento vendeu ingressos em cinco lotes diferentes. O número de ingressos vendidos por lote está no array `lotes = [50, 40, 60, 30, 70]`. Crie uma variável `totalIngressos` iniciando em 0. Use um laço `for` para percorrer o array e somar todos os valores na variável `totalIngressos`. Ao final, imprima "O total de ingressos vendidos foi: [total]".

Atividade 5:

Vamos trabalhar a parte lógica de vocês agora... Escreva um programa que verifique os números de 0 a 999 (essa vai ser a condição do for).

Desses, fazer a soma dos ímpares e pares e mostrar como saída (se $\text{numero \% 2} == 0$ é par).

Em seguida, mostrar também o total de números ímpares e pares.

Desafio: fazer a média aritmética da soma dos números ímpares e par, e imprimir qual dos dois teve o maior total.

Dicas: crie as variáveis: **somaPares**, **somaImpares**, **totalPares**, **totalImpares**, **mediaPares**, **mediaImpares**. Esse programa vai precisar de condicionais para incrementar as somas dentro do laço for, e para dizer qual das somas é maior. As médias podem ser calculadas como $\text{somaPares} / \text{totalPares}$ (o mesmo para ímpares). Para facilitar, use += para calcular as somas dos números, e ++ para calcular o total.

Switch

É uma condicional, alternativa ao uso de muitas condicionais como por exemplo:

```
if (variavel == 1) {  
    // acao  
} else if (variavel == 2 ) {  
    // acao2  
} else if (variavel == 3) {  
    // acao3  
} else {  
    // acaoGeral  
}
```



```
if (variavel == 1) {  
    // ação 1  
} else if (variavel == 2 ) {  
    // ação 2  
} else if (variavel == 3) {  
    // ação 3  
} else {  
    // acaoGeral  
}
```

```
switch (variavel) {  
    case 1:  
        // ação 1  
        break;  
    case 2:  
        // ação 2  
        break;  
    Case 3:  
        // ação 3  
    default:  
        // ação geral  
}
```

```
let cor = 'vermelho';
if (cor === 'vermelho') {
    console.log('Pare!');
} else if (cor === 'amarelo') {
    console.log('Atenção!');
} else if (cor === 'verde') {
    console.log('Siga!');
} else {
    console.log('Cor
desconhecida!');
}
```

```
let cor = 'vermelho';
switch (cor) {
    case 'vermelho':
        console.log('Pare!');
        break;
    case 'amarelo':
        console.log('Atenção!');
        break;
    case 'verde':
        console.log('Siga!');
        break;
    default:
        console.log('Cor desconhecida!');
}
```

Hora da Prática

Atividade 1:

Crie um programa que receba (via prompt-sync ou stdin) o gênero do filme escolhido: "Ação", "Comédia", "Terror" ou "Animação". Use a estrutura switch para imprimir em qual sala o filme passará:

- Ação: "Sala 1"
- Comédia: "Sala 2"
- Terror: "Sala 3"
- Animação: "Sala 4"
- Caso padrão (default): "Gênero não encontrado. Verifique as opções válidas."

Atividade 2:

Vamos desenvolver um programa que classifique notas, ele vai receber um número de 0 a 100 e usando switch, retorna uma String com a classificação da nota de acordo com o seguinte critério:

90 a 100: "A"

80 a 89: "B"

70 a 79: "C"

60 a 69: "D"

0 a 59: "F"

Caso Padrão: "Nota Inválida"

Dica: use como variável do switch `true`, e em cada caso, coloque condições to tipo `case (nota >= 90 && nota <= 100)` e assim por diante.

Desafio: pedir a nota para o usuário digitar usando `stdin`. E imprimir mensagem com a classificação e se ele foi aprovado (C para cima) ou não.

Atividade 3:

Crie uma variável `codigoProduto` e atribua a ela o valor "A1", "B2" ou "C3". Usando switch, verifique o código e imprima o nome do salgadinho escolhido:

- "A1": "Você escolheu: Batata Chips"
- "B2": "Você escolheu: Amendoim"
- "C3": "Você escolheu: Biscoito de Chocolate"
- Caso padrão (default): "Código inválido. Tente novamente."

Funções



Um bloco de código que agrupa instruções para realizar uma tarefa específica. Você escreve essas instruções uma única vez e pode "chamar" (reutilizar) esse bloco quantas vezes precisar, evitando a repetição de código.



Uma estrutura onde você insere dados (parâmetros), ela faz o processamento de forma isolada e te devolve um resultado pronto (retorno). É como uma caixa mágica que automatiza uma tarefa para você usar sempre que quiser.



Pense nela como uma mini-fábrica dentro do seu programa: você define como ela funciona uma vez, e depois é só enviar os materiais para ela fazer o trabalho pesado de forma automática sempre que você pedir.

Exemplo

```
function nomeDaFuncao(parametros) {  
    // código aqui  
}
```


Exemplo

```
function mostrarMensagem() {  
    console.log("Sistema iniciado com sucesso!")  
}
```

```
mostrarMensagem()
```

```
// Vai imprimir Sistema iniciado com sucesso!
```

Exemplo

```
function darBoasVindas(nomeUsuario) {  
    console.log("Bem-vindo(a), " + nomeUsuario + "!!")  
}
```

```
darBoasVindas("Ana")  
//Vai imprimir Bem-vindo(a), Ana!  
darBoasVindas("Carlos")  
//Vai imprimir Bem-vindo(a), Carlos!
```

Exemplo

```
function calcularDobro(numero) {  
    return numero * 2;  
}
```

```
let resultado = calcularDobro(15)  
console.log(resultado)  
// Vai imprimir 30
```


Exemplo de Arrow Function

```
const somar = (a, b) => {  
  return a + b;  
};
```

```
console.log(somar(10, 5))
```

Revisão

Como um programa funciona?



Entrada



Algoritmo



Saída



Algoritmo

o programa é essa sequência de passos...


```
/* O que precisamos, e onde vamos  
guardar esses valores? */  
let nome  
let idade
```




Criar uma conta do Google

Insira seu nome

Avançar


```
console.log('Como é seu nome?')
```




Criar uma conta do Google

Insira seu nome

Avançar


```
const prompt = require('prompt-sync')( )
```

```
nome = prompt( )
```

```
const prompt = require('prompt-sync')( )
```

```
nome = prompt( ).toString( )
```

```
const prompt = require('prompt-sync')( )
```

```
nome = prompt( ).toString( ).trim( )
```



```
console.log('Qual é sua idade?')
```

```
nome = parseInt(prompt( ).trim( ))
```



```
function nomeDaFuncao(argumento1, argumento2) {  
    // Código aqui  
}
```

```
function processamento(nome, idade) {  
    // Lógica aqui  
}
```

processamento(nome, idade)


```
const prompt = require('prompt-sync')( )
```

```
let nome
```

```
let idade
```

```
console.log('Qual é seu nome?')
```

```
nome = prompt( ).toString( ).trim( )
```

```
console.log('Qual é sua idade?')
```

```
idade = parseInt(prompt( ).trim( ))
```



```
if (idade < 18) {  
    let anosAteSerDeMaior = 18 - idade  
    console.log(`Olá ${nome}, você vai ser maior de idade em ${anosAteSerDeMaior} anos.`)  
} else {  
    console.log(`Olá ${nome}, você é maior de idade`)  
}
```



```
const prompt = require('prompt-sync')();
```

```
let nome = prompt('Como é seu nome? ');
```

```
let idade = parseInt(prompt('Qual é sua idade? '));
```

```
processamento(nome, idade);
```

```
function processamento(nome, idade) {
```

```
  if (idade < 18) {
```

```
    let anosAteSerDeMaior = 18 - idade;
```

```
    console.log(`Olá ${nome}, você vai ser maior de idade em ${anosAteSerDeMaior} anos.`);
```

```
  } else {
```

```
    console.log(`Olá ${nome}, você é maior de idade`);
```

```
  }
```

```
}
```



```
const prompt = require('prompt-sync')();

let nome = prompt('Como é seu nome? ');
let idade = parseInt(prompt('Qual é sua idade? '));

if (idade < 18) {
  let anosAteSerDeMaior = 18 - idade;
  console.log(`Olá ${nome}, você vai ser maior de idade em
  ${anosAteSerDeMaior} anos.`);
} else {
  console.log(`Olá ${nome}, você é maior de idade`);
}
```


Ou seja... nós usamos:



Variáveis: são elementos fundamentais que permitem armazenar e manipular dados em um programa. Elas representam locais na memória do computador onde valores podem ser armazenados e posteriormente recuperados ou modificados durante a execução do programa.



**nome e idade, iniciaram vazias, em seguida salvamos o
que o usuário digitou nelas (manipulação).**



Saída: para falar para o usuário o que ele precisa fazer, usamos o `console.log()`.



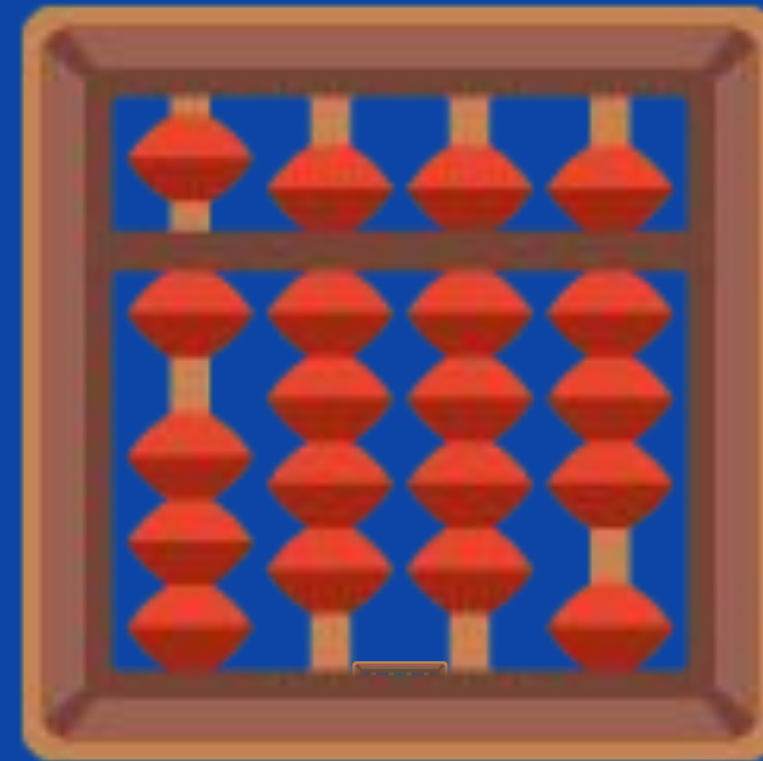
Entrada: para salvar o que usuário escreveu, usamos o `process.stdin`.



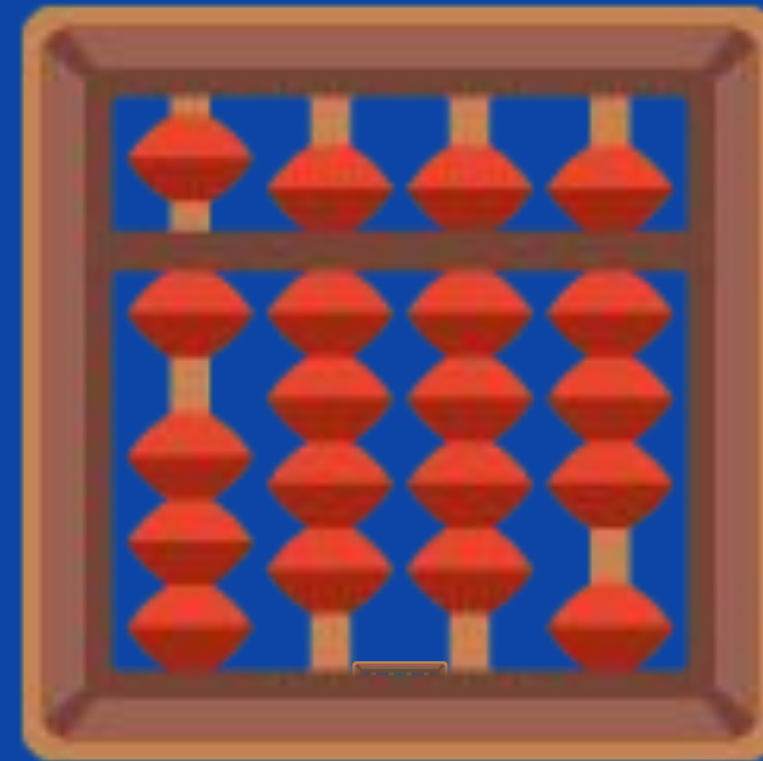
Função: para organizar melhor nosso código.



Condicional: para fazer o fluxo lógico do nosso programa - SE alguma coisa ENTÃO alguma coisa.



Operadores matemáticos: para calcular em quanto tempo ele será maior de idade.



Operadores lógicos: para escrever a condição (se idade é maior ou igual a 18).



Saída: fizemos a saída do nosso programa usando o `console.log()`, onde apresentamos pro usuário se ele é maior de idade ou menor, e em quanto tempo ele será maior.

Podemos encarar tudo isso como ferramentas. As ferramentas que usamos para ensinar o computador a fazer algo. Com elas podemos montar praticamente qualquer programa.

É como se fossem pecinhas de Lego.

Work

scratch.mit.edu

ScratchConfiguraçõesArquivoEditarTutoriaisInscreva-seEntrar

CódigoFantasiasSons

Movimento

Aparência

Som

Eventos

Controle

Sensores

Operadores

Variáveis

Meus Blocos

mude volume em -10

mude o volume para 100 %

volume

quando for clicado

quando a tecla espaço for pressionada

quando este ator for clicado

quando o cenário mudar para cenário1

quando ruído > 10

quando eu receber mensagem 1

transmita mensagem 1

transmita mensagem 1 e espere

Controle

quando for clicado

pergunte Qual o seu nome? e espere

mude nome para resposta

pergunte Qual a sua idade? e espere

mude idade para resposta

se idade < 18 então

digajunte Olá com nome

digavocê é menor de idade

senão

digajunte Olá com nome

digavocê é maior de idade

nome

idade

Você é menor de idade

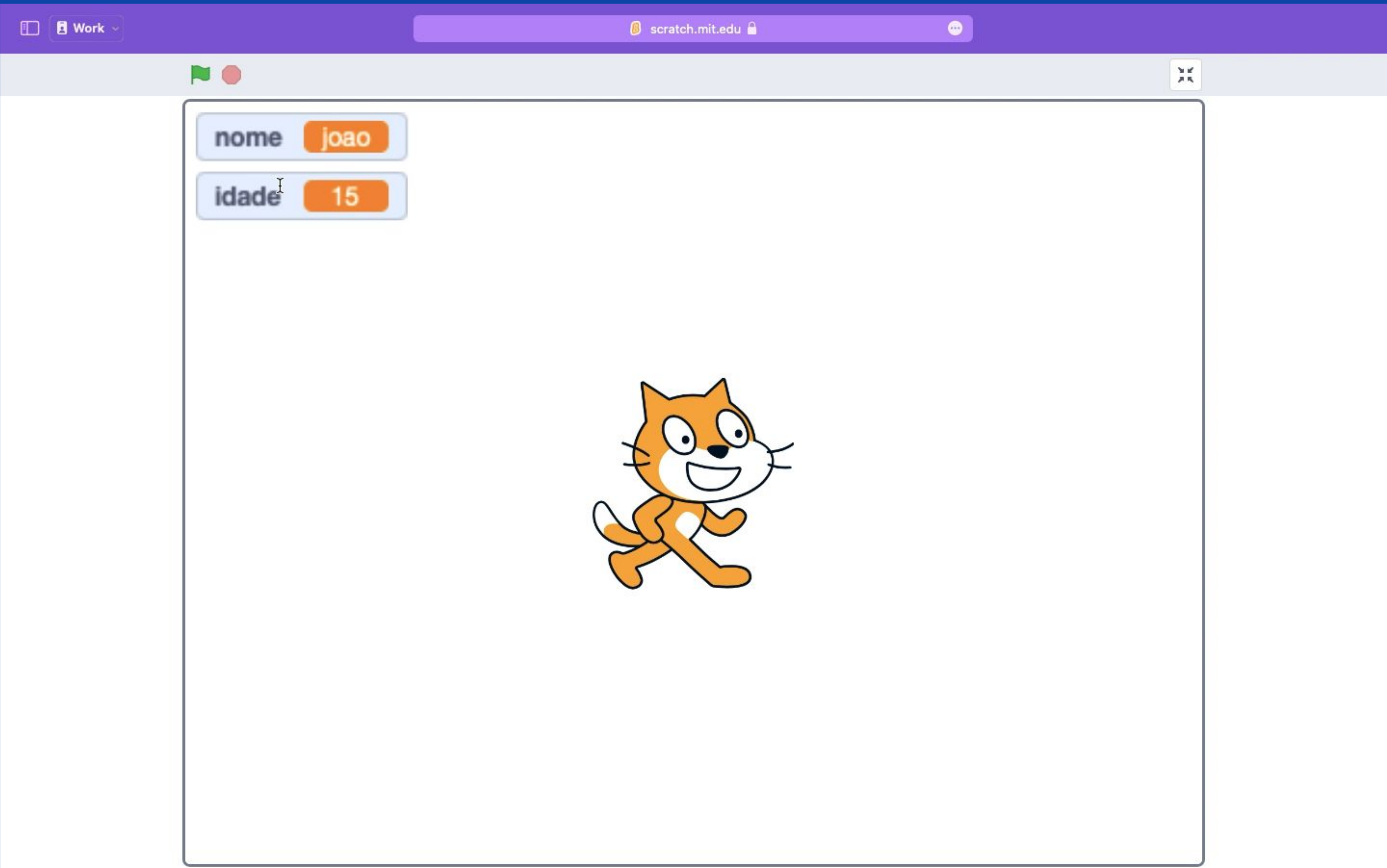
Ator1

x0y0

Ator1

Palco

Cenários1



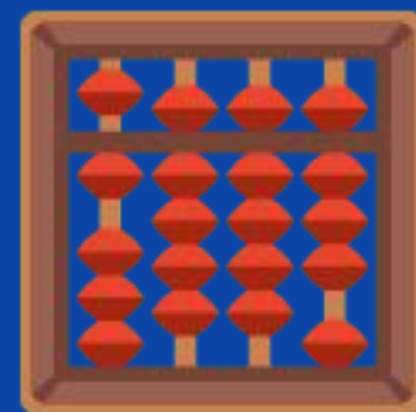
Quais são as principais ferramentas que vamos usar para montar nossos programas?



Variáveis



**Tipos de
Dados**



Operadores



Condicionais



Entrada



Saída



**Laços de
Repetição**



Funções

Variáveis

```
let,  
var,  
const
```

Tipos de Dados

```
string,  
int, float,  
boolean,  
array e  
objeto
```

Operadores

```
==, !=,  
>=, <=,  
++, --,  
...
```

Condiçõ es

```
if (...) {  
    ...  
} else {  
    ...  
}
```

Entrada

```
process.stdin  
prompt-sync
```

Saída

```
console.log()
```

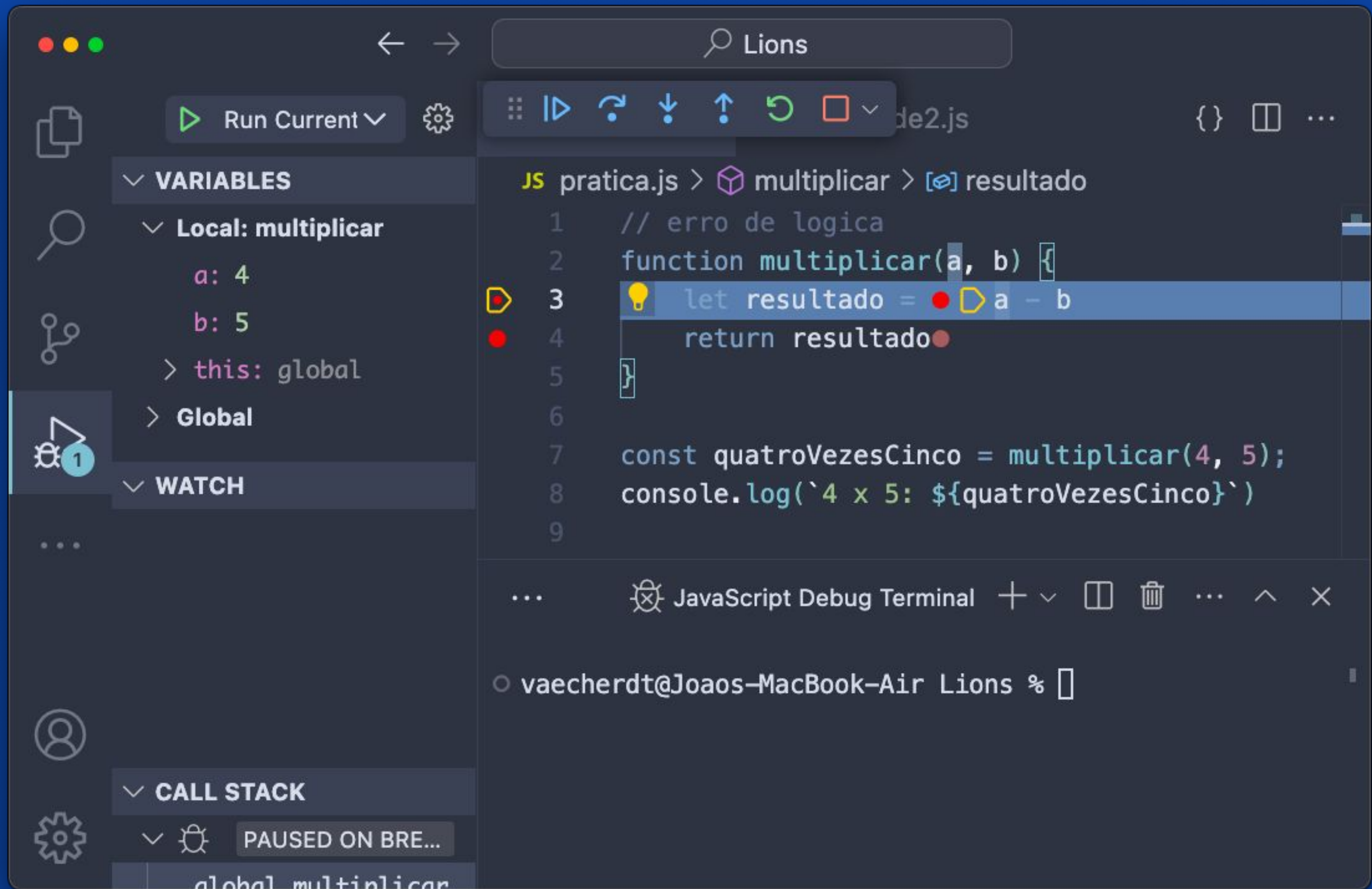
Laços de Repetição

```
for (...) {},  
while (...) {},  
do {} while  
(...)
```

Funções

```
function(..., ...)  
{  
    ...  
}
```

Debug



Run Current

Variables

Local: multiplicar

a: 4

b: 5

resultado: -1

Return value: -1

Watch

Call Stack

PAUSED ON BRE...

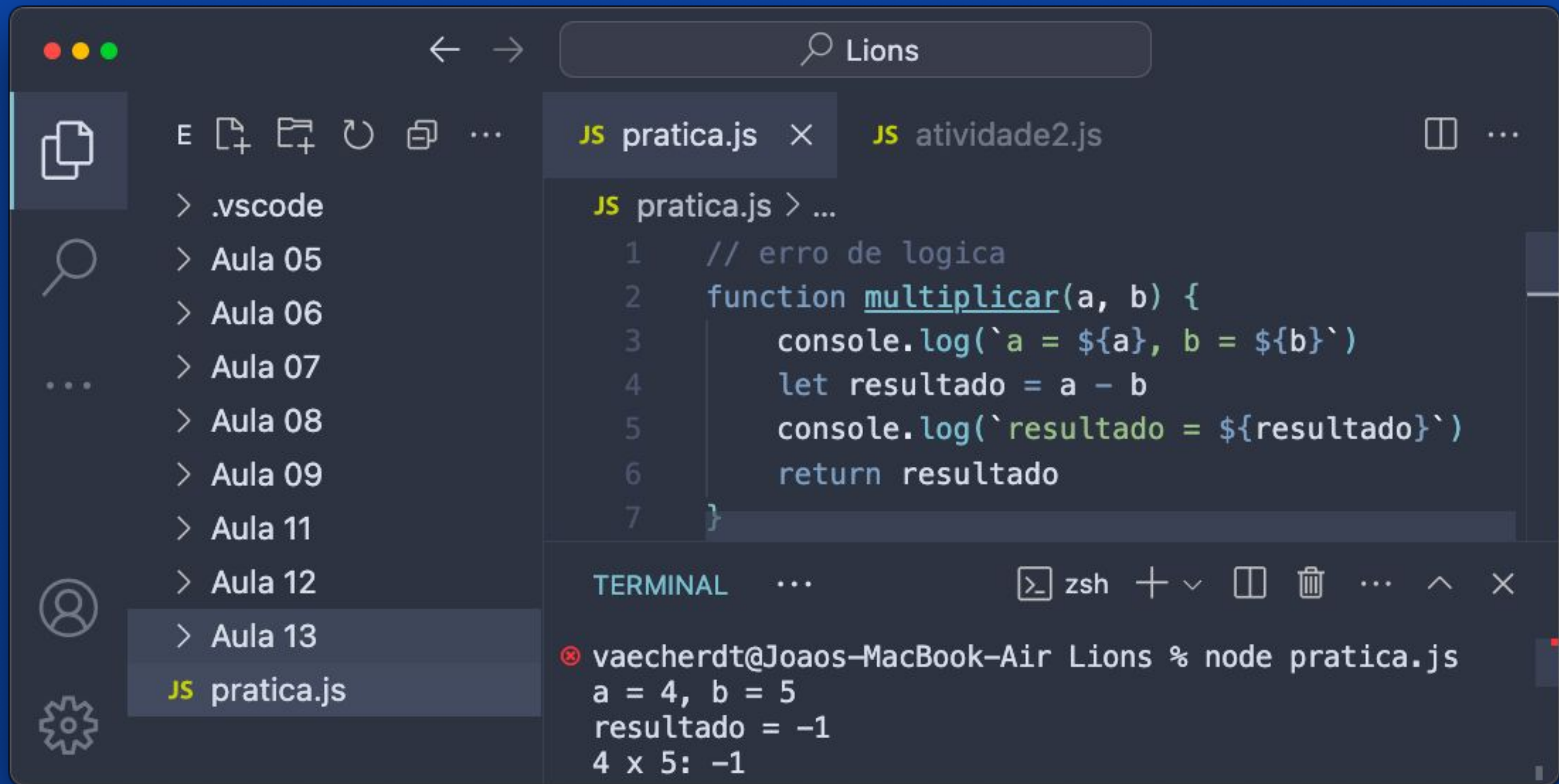
Lions

pratica.js > multiplicar

```
1 // erro de logica
2 function multiplicar(a, b) {
3   let resultado = a - b
4   return resultado
5 }
6
7 const quatroVezesCinco = multiplicar(4, 5);
8 console.log(`4 x 5: ${quatroVezesCinco}`)
9
```

JavaScript Debug Terminal

vaecherdt@Joaos-MacBook-Air Lions %



Exercícios de Revisão para Avaliação

Fim!